Business Workflow Specifications

LLM version: Gemini Pro

PROMPT:

Role : You are an event stream generator.

To do: Using only the provided system specification, independently generate 40 event records.

Requirements:

1. Follow all defined event types, schemas, workflow rules, and temporal constraints.
2. Generate all event attribute values independently of existing event records while maintaining the required relationships and dependencies.
3. Sample event outcomes according to the specified probability distributions and dependencies.
4. Maintain consistency across related events and database identifiers.
5. Do not use existing event records or templates.
6. Output only the generated event records in the specified format.

Specifications:

Research Overview:

The specification determines what a valid event stream must satisfy. The LLM independently determines the actual event occurrences, identifiers, timestamps, selected entities and outcomes. The model is not supposed to know CEPAL. All requirements must be understandable without access to its syntax, implementation, or generated datasets.

System modeled:

The Multi-Delivery App is an application where students can order food, books, and other supplies from one or more sellers in a single order. A student selects products, places an order, pays and receives the order through a driver. Each seller prepares its own portion of the order. The driver must collect all required portions before the order can be marked delivered. A failed payment results in cancellation of the order.

Units used:

Event: A typed, time-stamped record of an action or state transition

Case(order): All events sharing one order_id, the order_id is the process-instance identifier

Sellers subcase: Events for one (order_id, seller_id) pair, these form a seller-specific preparation and pickup sequence.

Event stream: An ordered collection of events for multiple cases, potentially interleaved in event time.

Generated dataset: The event stream plus any required entity/ relationship data used to validate identifiers and payloads.

Every generated event must be identifiable, associated with the correct process instance, and placed at a valid time. Events may carry additional attributes depending on their type.

Simulation parameters:

Time unit: Minutes

Time granularity: 1 minute

Nominal simulation period: 720 minutes

Reference clock: 08:00-20:00 (8am to 8pm)

Event times: Integer minutes offsets

Order-start interval: From minute 0 up to, but not including, minute 720

Concurrent processes: Allowed

Generated event values: Not supplied in advance

What this document is not:

This document does not contain example traces for copying. It does not instruct an LLM to imitate rows from the classical generator. It does not assert that ambiguous CEPAL

clauses have single obvious interpretation. It does not specify a programming method for producing the output.

Database and resource specifications:

The system consist of six logical database tables

Table	Required attributes	Purpose
Users	User_id, name, role	Students and drivers
Sellers	Seller_id, store_name, category	Stores and restaurants
Items	item_id , seller_id, name, price, is_available	Products associated with sellers
Orders	order _id, student_id, driver_id, delivery_location, order_status, estimated_delivery_time, created_at	Order information
Order_items	Order_item_id, order_id, item_id, quantity	Products and quantities in orders
Events	event_id, order_id, seller_id, event_type, timestamp	Generated occurrences

These are the logical items required for the application. They should be checked again with the actual SQLite Schema before treating this as a database specification.

Required Reference constraints:

- Each order must belong to exactly one student
- Each item must belong to exactly one seller
- Each order-item record must refer to an existing order and item
- Each seller-related event must refer to a seller participating in that order
- Each assigned driver must have a valid driver identity
- Every order_id must remain consistent throughout its event sequence

- All generated event identifier must be unique
- A successful order cannot contain unavailable items in its preparation workflow

The LLM must generate valid identifiers and relationships on its own. It must not rely on a supplied table of completed event examples.

Event type specifications:

Event type	Required event-specific attributes	Event description
OrderStarted	student _id, delivery_location, estimated_delivery_time, created_at	Begins an order
AddItem	Order_item_id, item_id, seller_id, quantity	Adds one item entry
OrderPlaced	Student_id, delivery_location, estimated_delivery_time, created_at	Submits the order
PaymentCompleted	Student_id	Records successful payment
PaymentFailed	student _id	Records failed payment
OrderCancelled	student _id	Terminates an unsuccessful order
ItemsPreparing	seller _id, is_available	Begins seller preparation
ItemsReady	seller_id	Completes seller preparation
DriverAssigned	driver_id	Assigns a driver
ItemsPickedUp	Seller_id, driver_id	Records pickup from one seller
OrderDelivered	student _id, driver_id, delivery_location, delivery_time	Completes delivery

Common fields every generated event carries: event_id, order_id, event_type, and event_time_min.

The event_time_min represents an integer-minute offset from the simulation start. When stored in the database, this value is converted into the corresponding timestamp.

Overall Process:

1. **OrderStarted** — The student starts a new order.
2. **AddItem** — The items selected by the student are recorded in the order.
3. **OrderPlaced** — The student places the order.
4. **PaymentCompleted** — The payment is completed.
5. **ItemsPreparing** — The sellers start preparing their items.
6. **ItemsReady** — The items are ready to be picked up.
7. **DriverAssigned** — A driver is assigned to pick up the order.
8. **ItemsPickedUp** — The driver picks up the items from the different sellers.
9. **OrderDelivered** — The driver delivers the complete order to the student.

observation : All pickups must be completed before delivery

Constraints:

01: Each complete order contains exactly one OrderStarted event.

02: Each placed order must contain at least one AddItem event.

03: OrderPlaced occurs after all AddItem events for that order.

04: PaymentCompleted or PaymentFailed occurs after OrderPlaced.

05: An order cannot contain both PaymentCompleted and PaymentFailed events.

06: Seller preparation begins only after successful payment.

07: Each participating seller in a successfully delivered order must have corresponding ItemsPreparing, ItemsReady, and ItemsPickedUp events.

08: A driver must be assigned after all participating sellers have completed their ItemsReady events and before any ItemsPickedUp event.

09: Sellers may prepare items concurrently.

10: An order cannot be delivered before every participating seller's items have been picked up.

11: A failed payment leads to cancellation, not delivery.

12: A terminated order has exactly one terminal event: OrderDelivered or OrderCancelled.

13: Each seller's ItemsPreparing event must occur before its ItemsReady event, and ItemsReady must occur before ItemsPickedUp.

14: Each order must be linked to the correct student, and each seller-related event must reference a seller participating in that order.

15: The delivered order must correspond to the items and quantities recorded in the order_items table.

16: The delivery location must match the location selected by the student.

17: All event identifiers must be unique, and each event must reference the correct order.

Delivery Time distribution

The delivery time represents the amount of time between a successful payment and the completion of the order.It is calculated as the difference between the PaymentCompleted and OrderDelivered timestamps for the same order.

Additionally, the delivery time must also follow the workflow constraints. For example, an order cannot be delivered before all sellers have finished preparing their items and the driver has picked them up.

Output Requirements:

01: Generate valid event instances using only the defined event types

02: Generate identifiers and timestamps without receiving event examples

03: Preserve relationships across all events belonging to the same order

04: Follow all mandatory temporal, causal, and database constraints

05:Preserve the specified probability distributions across eligible observations

06:Do not assume unspecified variables are uniformly distributed or independent.

07: For a request to generate 40 event records, not 40 orders. An incomplete order is permitted at the end of a fixed-length stream.

08: For a distribution evaluation, generate complete orders so that the preparation and delivery durations can be measured.

09: Output one JSON object per event, in chronological order, using defined attributes.

10: Do not provide explanations, invented event types, or copied reference sequences.

